Migrating a CircleCI Organization (GitHub OAuth → GitHub OAuth)

A step-by-step runbook with checklists and validation gates

CircleCI
2026

Migrating a CircleCI Organization

GitHub OAuth → GitHub OAuth

Use this guide when you are moving CircleCI configuration from one GitHub OAuth organization to another — for example, a GitHub org rename, moving to a new GitHub org, or consolidating two CircleCI orgs. Both organizations use the `gh/<name>` slug format.

Tooling: the `circleci-migrate` CLI.

Throughout, the **source** is `gh/acme` and the **destination** is `gh/acme-new`. Substitute your own slugs.

> **Safety first.** Nothing in the source org is ever modified. You can abort any time before applying the sync by simply stopping; the source keeps running normally.

What transfers

Transfers automatically	Requires a manual step
Contexts + context env vars	**Group** context restrictions (group IDs differ between orgs — recreate in UI)
Project-type context restrictions (remapped automatically via the project mapping)	SSH / checkout keys (re-add, or use `secrets transfer --include-ssh-keys`)
Project env vars	Webhook signing secrets (regenerate)
Project settings	Org orbs (republish in destination)
Org-level webhooks	Per-project access grants

Transfers automatically	Requires a manual step
Scheduled pipelines	
Self-hosted runner resource classes	

A complete “does not transfer” list is printed in your migration report (Phase 1).

The simplest path — guided `migrate`

For a first-time or one-off migration, the easiest way to start is the **guided interactive walkthrough**:

```
circleci-migrate migrate
```

Run it with no flags on an interactive terminal. It will prompt you for:

- Source and destination org slugs and API tokens
- Which sections to migrate (contexts, projects, org settings, extras, orbs, runners)
- Secrets are moved via the **in-pipeline transfer**

After gathering input it performs a **dry run, prints a concise summary**, and asks **“Apply these changes now?”** — answering yes applies without re-walking. After a successful apply, `migrate` **automatically runs the parity check** (equivalent to the `validate` command described in Phase 6).

`migrate` performs this exact ordering automatically — export → sync the structure (org settings, projects + project settings, contexts) → in-pipeline secrets transfer → post-apply validation — so the phase-by-phase flow below is simply the manual, explicit-control equivalent.

The phase-by-phase scripted flow described below remains the recommended path for operators who want explicit control, CI scripting, or the ability to pause and inspect between steps.

Phase 0 — Prerequisites

- [ ] Source slug confirmed: `gh/________` (Org Settings → Overview)
- [ ] Destination slug confirmed: `gh/________` , and the destination org **already exists** in CircleCI
- [ ] **Personal API tokens** for both orgs; each token’s user is an **org admin** of that org

- [ ] CLI installed and current:

```
brew install AwesomeCICD/tap/circleci-migrate
circleci-migrate version    # v0.17.1 or later
```

Set tokens for the session:

```
export CIRCLECI_SOURCE_TOKEN="<source-org-admin-token>"
export CIRCLECI_DEST_TOKEN="<destination-org-admin-token>"
```

If repositories also moved to a **different GitHub org**, also set a GitHub PAT with `repo` read: `export GITHUB_TOKEN="<pat>"`.

Preflight check (validates tokens, reachability, and common issues before you start):

```
circleci-migrate doctor --source-org gh/acme --dest-org gh/acme-new
```

✅ **Gate: all Phase 0 boxes checked, `doctor` reports no blockers.**

Phase 1 — Export and review (read-only)

Export never writes to CircleCI. It produces a manifest and a human-readable audit report.

```
circleci-migrate export \
  --source-org gh/acme \
  --output manifest.json \
  --report migration-report.md
```

If the source uses self-hosted runners, add `--runner-namespace <namespace>`.

Open `migration-report.md` and record your baseline counts:

Item	Count
Contexts	
Context env vars (total)	
Projects	
Project env vars (total)	
Webhooks	

Item	Count
Scheduled pipelines	

> **Tip — follow all projects:** if some source projects were never set up in CircleCI, they won't appear. The export will flag this and you can run `export --follow-all` (with a GitHub token) to onboard every repo first.

✅ **Gate: report reviewed in full; counts recorded; every warning understood.**

Phase 2 — Prepare the mapping

Create `mapping.json` now: `sync` uses it to remap project slugs to the destination, and the Phase 5 secrets transfer uses it to route project variables to the right destination project.

```
{
  "org": { "from": "gh/acme", "to": "gh/acme-new" }
}
```

If repos also moved GitHub orgs, add:

```
{
  "org": { "from": "gh/acme", "to": "gh/acme-new" },
  "github_org": { "from": "acme", "to": "acme-new" }
}
```

To match source projects to destination projects **by repository name** (needed for project env-var routing in Phase 5), generate the mapping instead:

```
circleci-migrate mapping generate --manifest manifest.json --dest-org gh/acme-new
        -o mapping.json
```

> `mapping generate` **needs the destination API token.** It reads the manifest and resolves destination project slugs against the live destination org, so it requires the **destination** token (`--dest-token` , or the `CIRCLECI_DEST_TOKEN` you already exported in Phase 0). For a **GitHub App** destination org, also pass `--github-token` (or set `GITHUB_TOKEN`).

✅ **Gate: `mapping.json` has the correct destination org.**

Phase 3 — Dry-run sync (structure)

Preview every change. Without `--apply`, nothing is written. This syncs the **structure** — org settings, projects + project settings, and contexts (context + variable **names** + restrictions) — **not** secret values.

```
circleci-migrate sync \
  --manifest manifest.json \
  --mapping mapping.json
```

> **Project-type context restrictions** are remapped automatically via `mapping.json` (source project UUIDs → destination project UUIDs). **Group** restrictions stay manual — group IDs differ between orgs and must be recreated in the destination UI after cutover.

Each line shows a status:

Status	Meaning
`would create`	Will be created on apply
`would set`	Value will be written
`exists`	Already present; reused
`manual`	Cannot be automated — handle in the UI
`error`	Investigate before applying

Review the dry run for `error` lines and "needs attention" / `manual` items.

> **Expected at this stage:** seeing "needs attention" for **project environment variables** and **context values** is normal and fine — those are secret *values*, which are never written by `sync`. They are moved later in **Phase 5 — Move secrets**, so they are **not** blockers here.

Only **unexpected** `error` **lines** or **non-secret** manual items (e.g. org orbs to republish, group context restrictions) need a plan before you apply.

✅ Gate: no unexpected `error` lines; every non-secret `manual` item has a plan; counts match Phase 1. ("Needs attention" on env vars / context values is expected — handled in Phase 5.)

Phase 4 — Apply sync (creates and follows projects)

```
circleci-migrate sync \
  --manifest manifest.json \
  --mapping mapping.json \
  --apply
```

Applying the sync creates the destination contexts and projects and **follows (enables) the destination projects** — for OAuth projects this installs the GitHub webhook so they begin receiving builds. No separate “enable builds” step is needed.

Re-run the dry-run command to confirm previously-`would create` lines now show `exists`.

After this phase the destination **projects and contexts exist** and the projects are following.

✅ **Gate: apply completed without errors; destination counts match; projects following.**

Phase 5 — Move secrets

Now that the destination projects and contexts exist (Phase 4), move the secret **values** into them. CircleCI never returns secret values via API, so values are moved from *inside* a pipeline — straight from source to destination over TLS, with nothing written to disk. **See the companion *Transferring Secrets Between CircleCI Organizations* guide** for the full walkthrough. In brief:

```
# Store the destination token in a SOURCE-org context named "migration-secrets"
# (variable: CIRCLECI_DEST_TOKEN). You already created mapping.json in Phase 2.

# Dry-run, then apply:
circleci-migrate secrets transfer --manifest manifest.json \
  --dest-org-id <dest-org-uuid> --dest-token-context migration-secrets \
  --mapping mapping.json --include-project-vars
circleci-migrate secrets transfer --manifest manifest.json \
  --dest-org-id <dest-org-uuid> --dest-token-context migration-secrets \
  --mapping mapping.json --include-project-vars --enable-trigger --apply
```

Context restrictions: project-type and expression restrictions are **remapped and recreated automatically** on the destination (the tool maps source project UUIDs to destination project UUIDs via `mapping.json`). **Group** restrictions are **not** transferred automatically — group IDs differ between orgs and must be recreated manually in the destination UI after cutover.

If any contexts have **project or expression restrictions** that block the transfer pipeline, the CLI fails fast with an actionable error. Add `--remove-restrictions` to temporarily lift them:

```
circleci-migrate secrets transfer --manifest manifest.json \
  --dest-org-id <dest-org-uuid> --dest-token-context migration-secrets \
  --mapping mapping.json --include-project-vars \
  --remove-restrictions --enable-trigger --apply
```

✅ **Gate: secrets transferred; restricted-context and SSH-key gaps noted.**

Phase 6 — Validate

Run the `validate` command as the primary parity check — it exports both orgs read-only and prints a per-section report covering Contexts, Projects, Org Settings, Runners, Orbs, and CIAM. Exit code is non-zero if any item is **missing** from the destination.

```
circleci-migrate validate \
  --source-org gh/acme \
  --dest-org gh/acme-new \
  --mapping mapping.json
```

If orbs or runners were migrated, add the destination namespaces so those sections are checked (omitting them causes the section to be skipped with a note):

```
circleci-migrate validate \
  --source-org gh/acme \
  --dest-org gh/acme-new \
  --mapping mapping.json \
  --dest-orb-namespace acme-new \
  --dest-runner-namespace acme-new
```

The report shows:

Symbol	Meaning
✓	Matched — present and correct on the destination
×	Missing — needs action
⚠	Manual — requires operator action; does not fail the exit code

A **NEEDS ATTENTION** block at the end lists every × missing and ⚠ manual item.

Expected ⚠ manual items:

- **Group context restrictions** — group IDs differ between orgs; recreate in the destination UI after cutover.
- **SSO** — DNS verification and IdP setup are always manual.

- **CIAM** — role bindings must be confirmed by email.
- A freshly-followed project may briefly appear as "not followed" until the follow propagates — re-run `validate` a minute later if you see this.

> **Note:** secret *values* are never compared — the CircleCI API masks them. `validate` checks presence and structure (names, counts, settings, restrictions).

After validate passes (exit 0), work through any ⚠ items listed (e.g. recreate group context restrictions, regenerate webhook signing secrets, confirm CIAM role bindings). You can also spot-check in the destination UI — confirm a pipeline runs green and a job can read its context variables.

Org feature flags — "danger" items

Two org feature flags are **skipped by default** for safety: `drop_all_build_requests` and `require_context_group_restriction`. Enabling either on a fresh org can freeze or break pipelines. When the source org has either set to `true`, the migration report surfaces them as a manual step. Once the destination is validated and stable, write them explicitly:

```
circleci-migrate sync \
  --manifest manifest.json \
  --mapping mapping.json \
  --include-danger-flags \
  --apply
```

✅ **Gate: `validate` passed (exit 0); every ⚠ manual item triaged with a plan.**

Phase 7 — Post-cutover

- [ ] Rotate the **destination API token** used for the secrets transfer once it is complete.
- [ ] Repoint external references to the new org: status badges, Slack/ notifications, dashboards, Backstage/service catalog, and **GitHub branch-protection required status checks**.
- [ ] **On the SOURCE org, turn on "disable new builds"** once traffic has moved to the destination. This is the org-level pause-new-builds control (`drop_all_build_requests` , in Organization Settings) — flipping it on stops the old org from running builds during/after cutover. It is a safe, **reversible** step: turn it back off if you need to fall back to the source while validating the destination.
- [ ]

Decommission the source org when confident: unfollow its projects (removes webhooks), archive, and notify your team.

✅ **Gate: destination token rotated; new builds disabled on the source; external pins updated; team notified.**

Troubleshooting

Symptom	Fix
`manual` on group context restrictions	Group IDs differ between orgs; recreate group restrictions in the destination UI.
`manual` on project context restrictions	Should transfer automatically if `mapping.json` has a matching entry; verify the source project slug is in the mapping.
`manual` / "needs attention" on env vars	Expected — secret values move in Phase 5 via `secrets transfer`. Use `--missing-secrets placeholder` if you want the names created for manual fill-in.
Project missing from the plan	Destination can't follow it yet — check GitHub access / onboarding.
Pipeline skipped during secrets transfer	Enable "unversioned config" in the source org (or pass `--enable-trigger`).
Builds didn't start after apply	Confirm the project is *following* and the webhook installed on GitHub.

Command quick reference

```
# Guided interactive walkthrough (simplest):
circleci-migrate migrate

# Or, step-by-step:
circleci-migrate doctor   --source-org gh/acme --dest-org gh/acme-new
circleci-migrate export   --source-org gh/acme --output manifest.json --report
        migration-report.md
circleci-migrate mapping generate --manifest manifest.json --dest-org gh/acme-new
        -o mapping.json
circleci-migrate sync --manifest manifest.json --mapping
        mapping.json                # dry run (structure)
circleci-migrate sync --manifest manifest.json --mapping mapping.json --
        apply         # create + follow projects
# secrets: in-pipeline transfer (see the Secrets Transfer guide):
```

```
circleci-migrate secrets transfer --manifest manifest.json \
  --dest-org-id <dest-org-uuid> --dest-token-context migration-secrets \
  --mapping mapping.json --include-project-vars --enable-trigger --apply
circleci-migrate validate --source-org gh/acme --dest-org gh/acme-new --mapping
        mapping.json
```

© CircleCI. Generated for customer use. Commands assume `circleci-migrate` v0.17.1 or later. Nothing in this runbook modifies the source organization.